به نام خدا

علی سپهرنیا (۹۶۱۲۱۰۳۸) گزارش کار آزمایشگاه سیستم دیجیتال ۲ (جلسه هفتم) استاد: خانم رضایی

۱- آزمایش ۱

برنامه ای بنویسید که اگر تعداد پالس های ورودی به واحد کانتر میکرو به ۱۰ رسید یک آلارم دهد.

شرح گزارش کار :

I. در کدویژن یک پروژه جدید ایجاد میکنیم. در پنجره کد ویزارد چیپ ATMEGA16 را انتخاب میکنیم. پایه PORTC0 را برای وصل کردن LED ، خروجی تعریف می کنیم. به قسمت Timers > Timer0 میرویم. چون از کلاک خارجی استفاده می کنیم Clock Source را روی T0 Pin Rising Edge میگذاریم. Mode را روی CTC می گذاریم. چون موج خروجی نمیخواهیم پس Output را برابر Disconnected قرار می دهیم. می خواهیم با هر ۱۰ بار شمارش به ما هشدار دهد پس حالت Compare Match Interrupt را فعال میکنیم و Compare را برابر 9 قرار می دهیم. بقیه تنظیمات را تغییر نمی دهیم. مطمئن میشویم تیک گزینه Program > Generate Code For Disabled Peripherals برداشته شده است. سپس گزینه Program > Generate, Save and Exit را میزنیم. در سه پنجره باز شده برای ذخیره کردن فایل های پروژه همه ی فایل ها را در یک فولدر و در کنار هم ذخیره میکنیم.

II. تابع timer0_comp_isr به طور خودکار ایجاد می شود. این تابع بعد از هر ۱۰ پالس ورودی به واحد کانتر، فراخوانی می شود. بنابراین باید کد مربوط به روشن و خاموش شدن LED را در این تابع بنویسیم. رجیستر های مربوط به تایمر به طور خودکار تنظیم شده اند بنابراین نیازی به کنترل آن ها نیست. داخل تابع تایمر، خروجی PORTC.0 را ۱ میکنیم و بعد از delay_ms(100) دوباره ۰ میکنیم. از منوی Project > Build All برنامه را کامپایل میکنیم.

نحوه کار برنامه به این صورت است که بعد از هر ۱۰ بار شمارش پالس، LED چشمک می زند.

III. در پروتئوس یک پروژه جدید با تنظیمات پیش فرض ویزارد ایجاد میکنیم. LED، میکروکنترلر ATMEGA16، مقاومت و اسلیوسکوپ را به پروژه اضافه میکنیم. LED را با کمک مقاومت محدودکننده جریان به PC0 و از آن طرف به زمین متصل میکنیم. یک منبع کلاک با سطح 5V، فرکانس 1Hz و دیوتی سایکل 50% را جهت اعمال پالس به پایه T0 متصل میکنیم. برای مشاهده تغییرات، پایه T0 و PC0 را به اسلیوسکوپ نیز متصل میکنیم. روی میکرو دابل کلیک میکنیم. در پنجره باز شده بخش Program File کد هگز برنامه تولید شده در کدویژن را روی میکرو پروگرم میکنیم. با اجرای شبیه ساز مشاهده میکنیم که بعد از ۱۰ بار شمارش پالس، LED چشمک می زند.

۲- آزمایش ۲

برنامه ای بنویسید که دور موتور DC را با استفاده از مولد PWM کنترل کند.

شرح گزارش کار :

I. در کدویژن یک پروژه جدید ایجاد میکنیم. در پنجره کد ویزارد چیپ ATMEGA16 را انتخاب میکنیم. به قسمت Timers > Timer0 میرویم. چون از کلاک داخلی استفاده می کنیم Clock Source را روی System Clock میگذاریم. همچنین مقدار کلاک را برابر 3.906KHz قرار میدهیم. Mode را روی Fast PWM می گذاریم. چون موج خروجی به صورت صحیح (نه معکوس) میخواهیم پس Output را برابر Non-Inverted PWM قرار می دهیم. بقیه تنظیمات را تغییر نمی دهیم. مطمئن میشویم تیک گزینه Program > Generate Code For Disabled Peripherals برداشته شده است. سپس گزینه Program > Generate, Save and Exit را میزنیم. در سه پنجره باز شده برای ذخیره کردن فایل های پروژه همه ی فایل ها را در یک فولدر و در کنار هم ذخیره میکنیم.

II. رجیستر های مربوط به تایمر به طور خودکار تنظیم شده اند بنابراین نیازی به کنترل آن ها نیست. رجیستر OCR0 را برابر 128 قرار میدهیم. با این کار دیوتی سایکل برابر 50% می شود. از منوی Project > Build All برنامه را کامپایل میکنیم.

III. در پروتئوس یک پروژه جدید با تنظیمات پیش فرض ویزارد ایجاد میکنیم. LED، میکروکنترلر ATMEGA16، مقاومت، دیود، ترانزیستور NPN، خازن و اسلیوسکوپ را به پروژه اضافه میکنیم. اتصالات را مطابق شماتیک پیوست آزمایش ۲ انجام میدهیم. روی میکرو دابل کلیک میکنیم. در پنجره باز شده بخش Program File کد هگز برنامه تولید شده در کدویژن را روی میکرو پروگرم میکنیم. با اجرای شبیه ساز مشاهده میکنیم که بعد از تعادل، موتور با ولتاژ 7v~ و سرعت 300rpm~ کار میکند.

پیوست ها:

آزمایش ۱)

G:\Personal\Uni\Micro Lab\Micro\Az7-1\Az7-1.c

Notes | Az7-1.c

```
/*****************************************************
This program was created by the
CodeWizardAVR V3.12 Advanced
Automatic Program Generator
© Copyright 1998-2014 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project : Session7-Experiment1
Version :
Date    : 11/4/2020
Author  : Ali Sepehrnia (96121038)
Company :
Comments:


Chip type               : ATmega16
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 256
*****************************************************/

#include <mega16.h>
#include <delay.h>

// Declare your global variables here

// Timer 0 output compare interrupt service routine
interrupt [TIM0_COMP] void timer0_comp_isr(void)
{
    PORTC.0=1;
    delay_ms(100);
    PORTC.0=0;
}
```

G:\Personal\Uni\Micro Lab\Micro\Az7-1\Az7-1.c

Notes | Az7-1.c

```
void main(void)
{
// Declare your local variables here

// Input/Output Ports initialization
// Port A initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

// Port B initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (0<<DDB4) | (0<<DDB3) | (0<<DDB2) | (0<<DDB1) | (0<<DDB0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

// Port C initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (1<<DDC0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

// Port D initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

// Timer/Counter 0 initialization
// Clock source: T0 pin Rising Edge
// Mode: CTC top=OCR0
// OC0 output: Disconnected
TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (1<<WGM01) | (1<<CS02) | (1<<CS01) | (1<<CS00);
TCNT0=0x00;
OCR0=0x09;
```

G:\Personal\Uni\Micro Lab\Micro\Az7-1\Az7-1.c

Notes | Az7-1.c

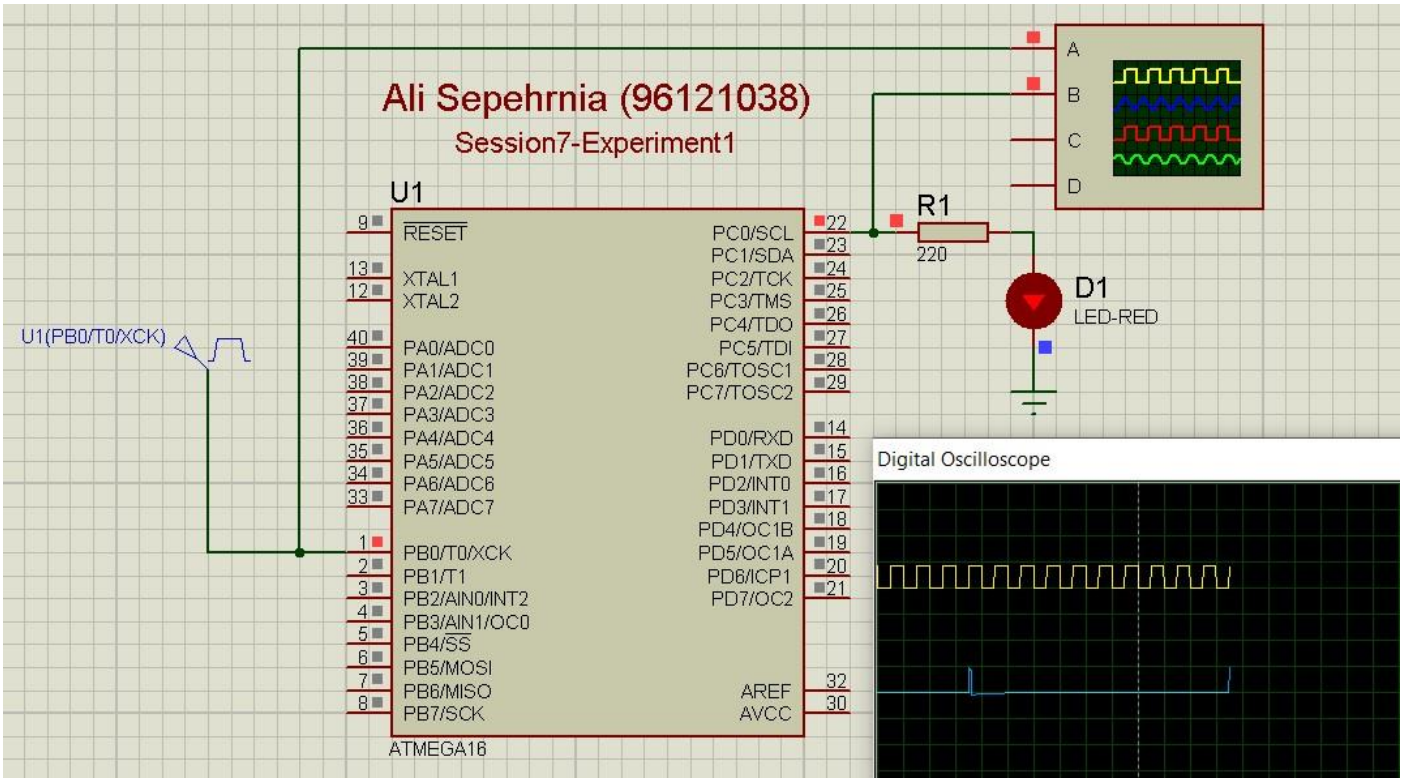
```
// Timer/Counter 0 initialization
// Clock source: T0 pin Rising Edge
// Mode: CTC top=OCR0
// OC0 output: Disconnected
TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (1<<WGM01) | (1<<CS02) | (1<<CS01) | (1<<CS00);
TCNT0=0x00;
OCR0=0x09;

// Timer(s)/Counter(s) Interrupt(s) initialization
TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (1<<OCIE0) | (0<<TOIE0);

// Global enable interrupts
#asm("sei")

while (1)
      {
      // Place your code here

      }
}
```



آزمایش ۲)

G:\Personal\Uni\Micro Lab\Micro\Az7-2\Az7-2.c

Notes Az7-2.c

```
/*****************************************************
This program was created by the
CodeWizardAVR V3.12 Advanced
Automatic Program Generator
© Copyright 1998-2014 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project : Session7-Experiment2
Version :
Date    : 11/4/2020
Author  : Ali Sepehrnia (96121038)
Company :
Comments:


Chip type               : ATmega16
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 256
*****************************************************/

#include <mega16.h>

// Declare your global variables here

void main(void)
{
// Declare your local variables here

// Input/Output Ports initialization
// Port A initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
```

G:\Personal\Uni\Micro Lab\Micro\Az7-2\Az7-2.c

Notes Az7-2.c

```
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

// Port B initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=Out Bit2=In Bit1=In Bit0=In
DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (0<<DDB4) | (1<<DDB3) | (0<<DDB2) | (0<<DDB1) | (0<<DDB0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=0 Bit2=T Bit1=T Bit0=T
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

// Port C initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

// Port D initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

// Timer/Counter 0 initialization
// Clock source: System Clock
// Clock value: 3.906 kHz
// Mode: Fast PWM top=0xFF
// OC0 output: Non-Inverted PWM
// Timer Period: 65.536 ms
// Output Pulse(s):
// OC0 Period: 65.536 ms Width: 0 us
TCCR0=(1<<WGM00) | (1<<COM01) | (0<<COM00) | (1<<WGM01) | (1<<CS02) | (0<<CS01) | (0<<CS00);
TCNT0=0x00;
OCR0=128;

// Timer(s)/Counter(s) Interrupt(s) initialization
TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
```

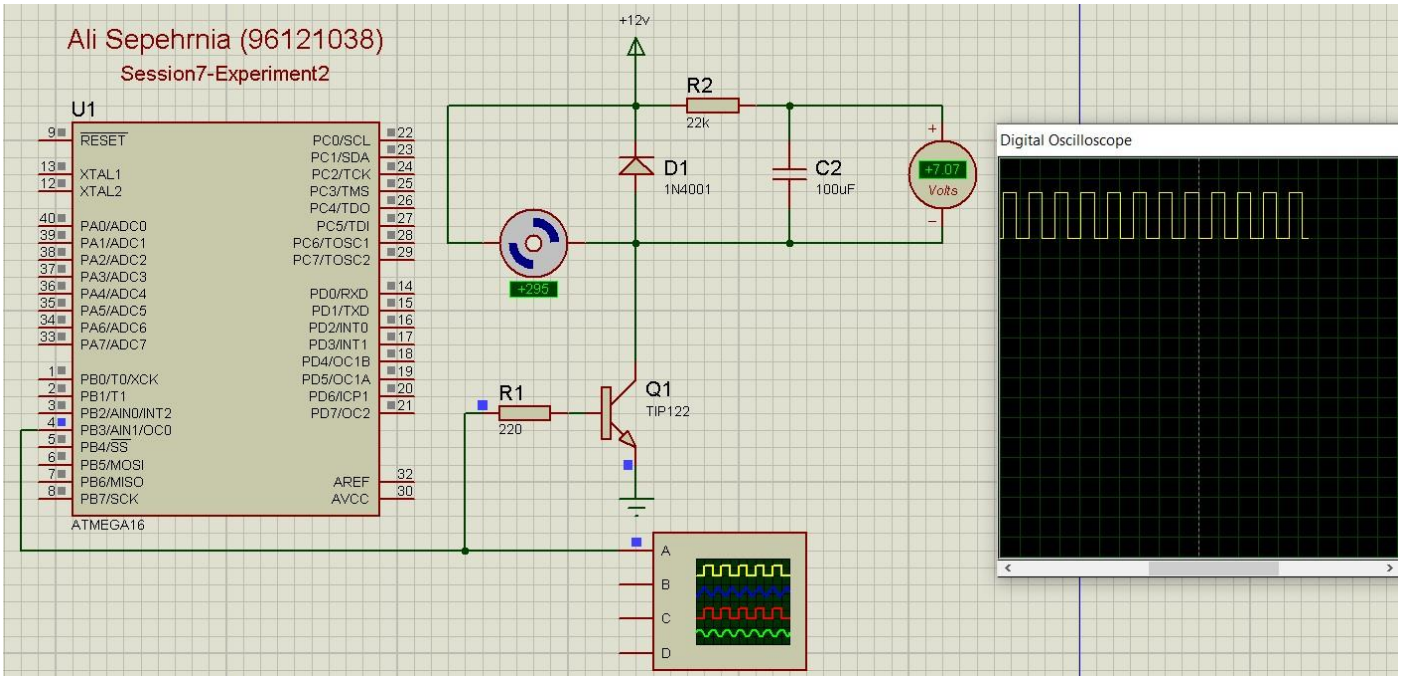
G:\Personal\Uni\Micro Lab\Micro\Az7-2\Az7-2.c

Notes Az7-2.c

```
// Timer(s)/Counter(s) Interrupt(s) initialization
TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);

while (1)
      {
      // Place your code here

      }
}
```


Ali Sepehrnia (96121038)
Session7-Experiment2
U1
ATMEGA16
+12v
R2
22k
D1
1N4001
C2
100uF
+7.07
Volts
+295
R1
220
Q1
TIP122
Digital Oscilloscope